



Runtime Debugger Toolkit - User Manual

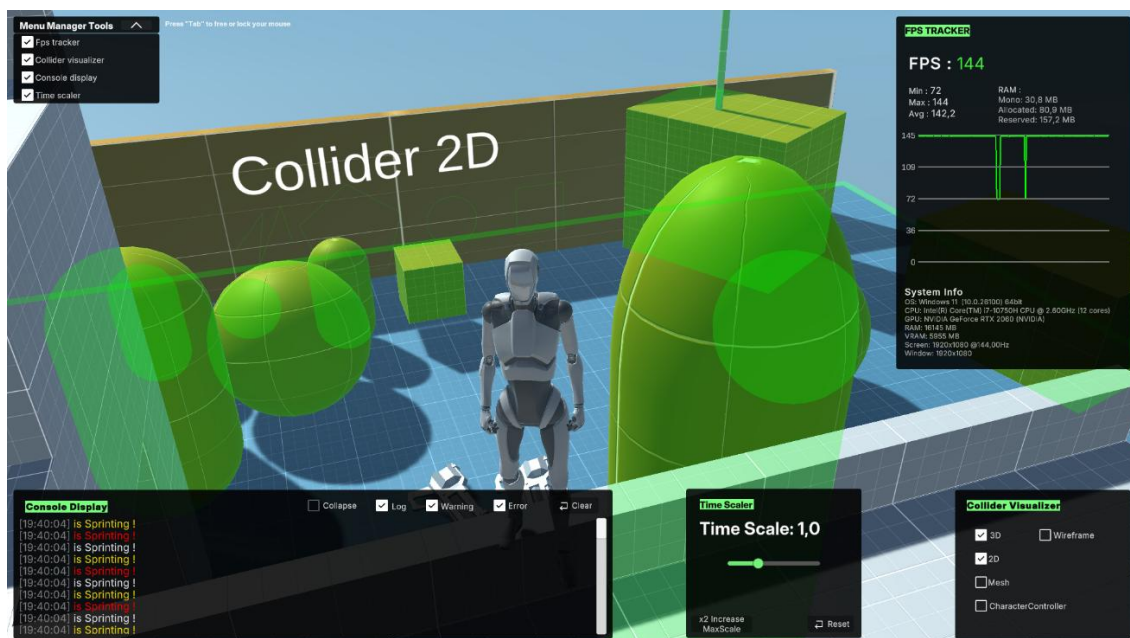
Introduction

Asset Purpose

This asset is designed to support debugging during Unity project development. It consolidates several runtime tools into a single, easy-to-integrate package:

1. **Collider Visualizer** – Visually displays 2D and 3D colliders in-game, helping you understand physical interactions at runtime.
2. **In-Game Console** – Displays Unity's log messages (Logs, Warnings, Errors) within the game window, both in the editor and in builds.
3. **FPS Tracker** – Tracks FPS in real-time via a live graph, while also providing memory usage and detailed system info (OS, resolution, processor, RAM, etc).
4. **Time Scaler** – Allows simulation speed adjustment to help with debugging, slow-motion analysis, or performance tuning.

Each tool is accessible via draggable in-game UIs, giving you flexible control over your simulation and making bug identification and performance tuning much easier.





Installation

Import the asset via the Unity Package Manager after purchasing it from the Unity Asset Store.

If this is your first time using it, we recommend importing the Demo folder, which includes a playground scene built on Unity's Starter Asset (Third Person Controller). For the demo scene to work, make sure you also import **Cinemachine**, **Input System** and **TextMeshPro** packages.

If you prefer not to use the demo, simply exclude the Demo folder and drag the `DebuggerToolManager.prefab` into your scene. Make sure you have a **UI Event System** present in your scene to enable interaction with the tools.

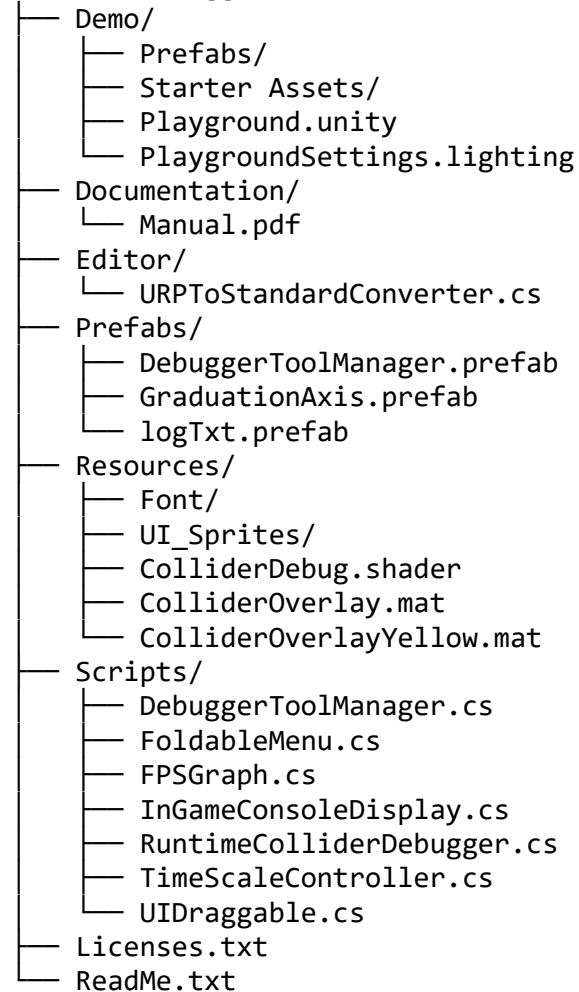
Supported Unity Version:

Originally developed on **Unity 2022.3.7f1**, tested in URP, Built-in, and HDRP. The tools work in all pipelines but note that the **Collider Visualizer is not compatible with HDRP**.



Folder Structure

Runtime Debugger Toolkit/





Folder Descriptions:

- Demo/ – Includes a sample playground scene to test and interact with the debugger tools. Optional for use.
 - Documentation/ – Includes Debugger Tool Manual.pdf
 - Prefabs/ – Core UI prefabs, especially DebuggerToolManager.prefab.
 - Resources/ – Includes fonts, sprites, shaders, and materials used in visualizations.
 - Scripts/ – All runtime scripts powering the tools.
 - Editor/ – Contains utility for converting URP materials to Built-in.
-

Usage

Running the Demo

In URP and HDRP, materials work out of the box. For Standard pipeline, use the converter script `URPToStandardConverter.cs` under `Tools > URPToStandardConverter` to adapt materials.

Prefab Setup

The main prefab is `DebuggerToolManager`. Its `DebuggerToolManager.cs` script handles mouse lock/unlock (default key: Tab) to enable interaction with in-game UIs. If this key is already used in your project, you can disable this behavior in the Inspector and change the key inside the script.

Inside this prefab, only the `FoldableMenu` is active by default. It allows toggling visibility of the 4 tools independently. The script `DebuggerToolManager.cs` dynamically shows/hides UI based on these toggles.



Tool Details

1. Collider Visualizer - Script: RuntimeColliderDebugger - Visualizes different collider types (2D/3D). Color meaning : - Green = accurate physical mesh - Yellow = approximated convex mesh (Unity's internal algorithms are not publicly accessible). You can choose between Wireframe or solid color mode. - Colors can be customized via Inspector.

2. In-Game Console - Script: InGameConsoleDisplay.cs on InGameConsole_Panel - Toggle log types (Info, Warning, Error) - Adjustable max message number inside the inspector (default: 100)

3. FPS Tracker - Script: FPSGraph.cs on Graph_Panel - Shows FPS graph and system stats – You can adjust graph size and choose between dynamic value of the max fps or cap it inside this inspector with the “ShowDynamicMaxFPS” boolean which enables/disables automatic scaling allowing to see with ease if fps are going under the max value defined.

4. Time Scaler - Script on Time_Scaler_Canvas - In-game slider to adjust Time.timeScale - Button “x2 Increase MaxScale” multiply by two the value of the slider upper value. (no upper limit) - Reset button restores scale to 1.0 and default range (0.1–3.0)

Customization

The tools are designed to be modular and extensible. Scripts are clearly structured, with many exposed Inspector variables for easy tweaking. Developers can adapt or expand the functionality as needed.



Tips

If you want to render imported meshes (e.g., .fbx, .obj files) in **wireframe mode** at runtime **within a build** you must enable **Read/Write** access on those meshes.

Otherwise, Unity will throw the error:

"Not allowed to access vertices on mesh..."

To enable Read/Write access:

1. Select the mesh asset in the **Project window** (inside the Assets folder).
2. In the **Inspector**, navigate to the **Model** tab.
3. Check the box labeled **Read/Write Enabled**.
4. Click **Apply** to save the changes.

This ensures Unity allows vertex data access at runtime, which is required for custom rendering such as wireframe overlays.

Contact

If you encounter issues or want to provide feedback:

 **ThornyDevStudio@gmail.com**

Thank you for using *Runtime Debugger Toolkit* ! 🎮